

Deployment using Hugging Face Spaces

Procedure in a nutshell

1. Download the hotel-assistant folder zip file and unzip it
2. Create a new GitHub Codespace using the Blank template
3. Upload the hotel-assistant folder on GitHub Codespace
4. Build a Docker Image
5. Run a Docker Container
6. Login to Docker Hub
7. Push the Image to Docker hub
8. Create a new Hugging Face Space repository
9. Create a Dockerfile to use the Docker Image present in Docker Hub
10. Build and test the application

Hugging Face Spaces

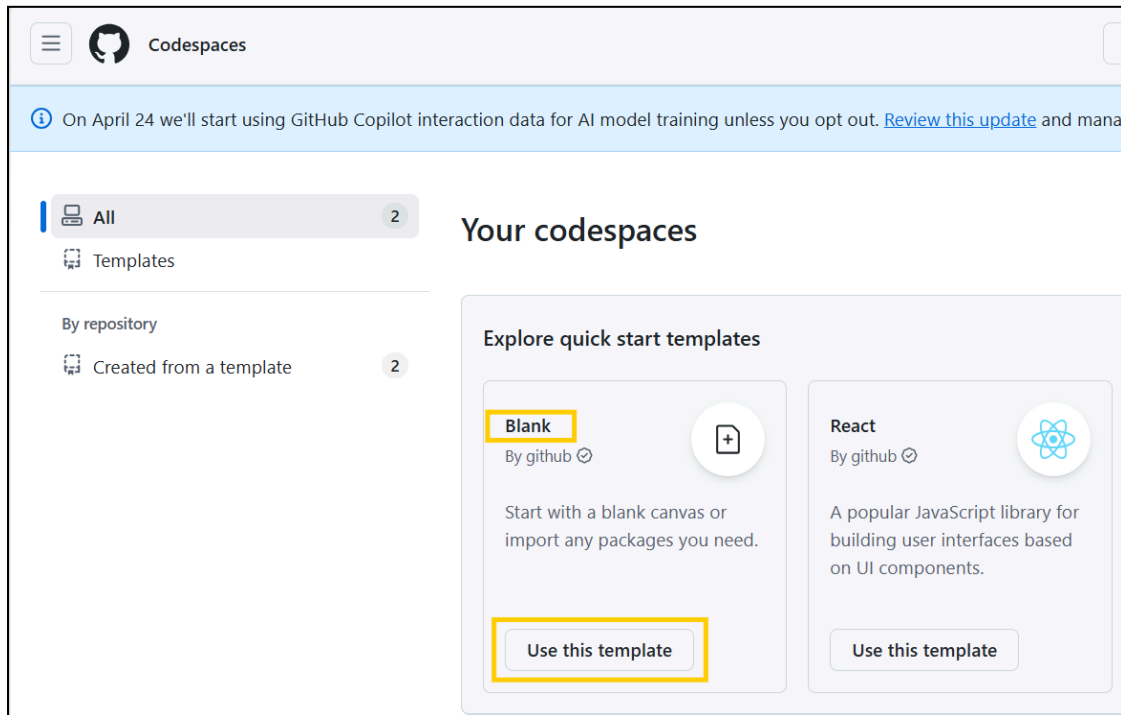
Spaces

- A resource that allows you to host and share machine learning apps
- Building a demo is one of the best ways to showcase and test what you've done
- Spaces allows you to do just that
- Many amazing AI apps made by the community: <https://huggingface.co/spaces>

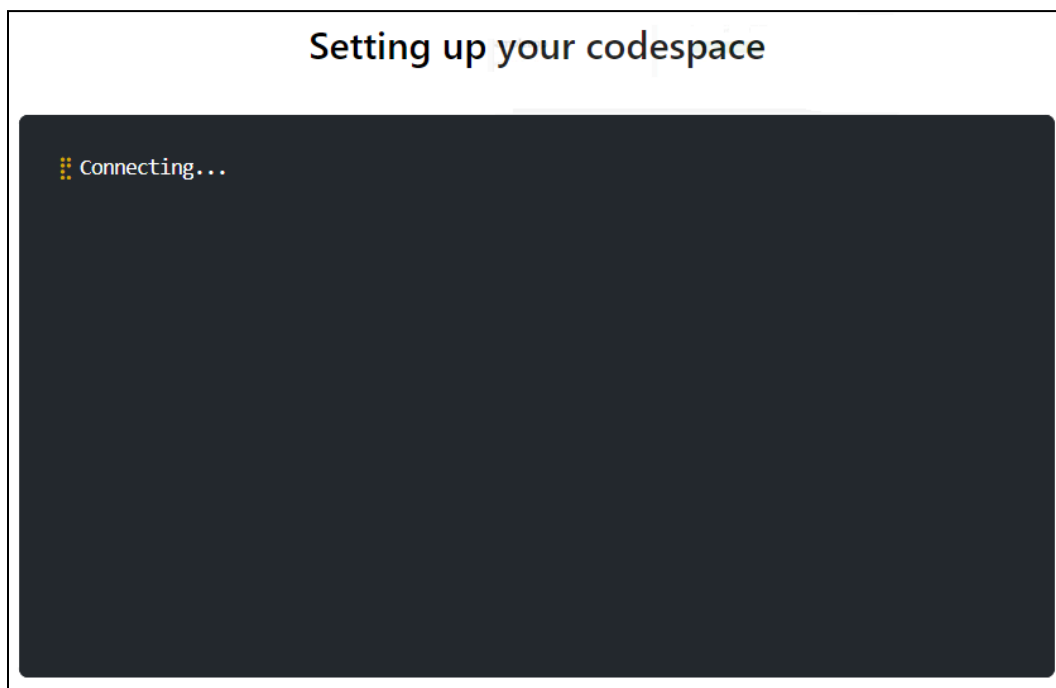
For more details on HF Spaces, refer to the [Spaces Overview](#).

Steps

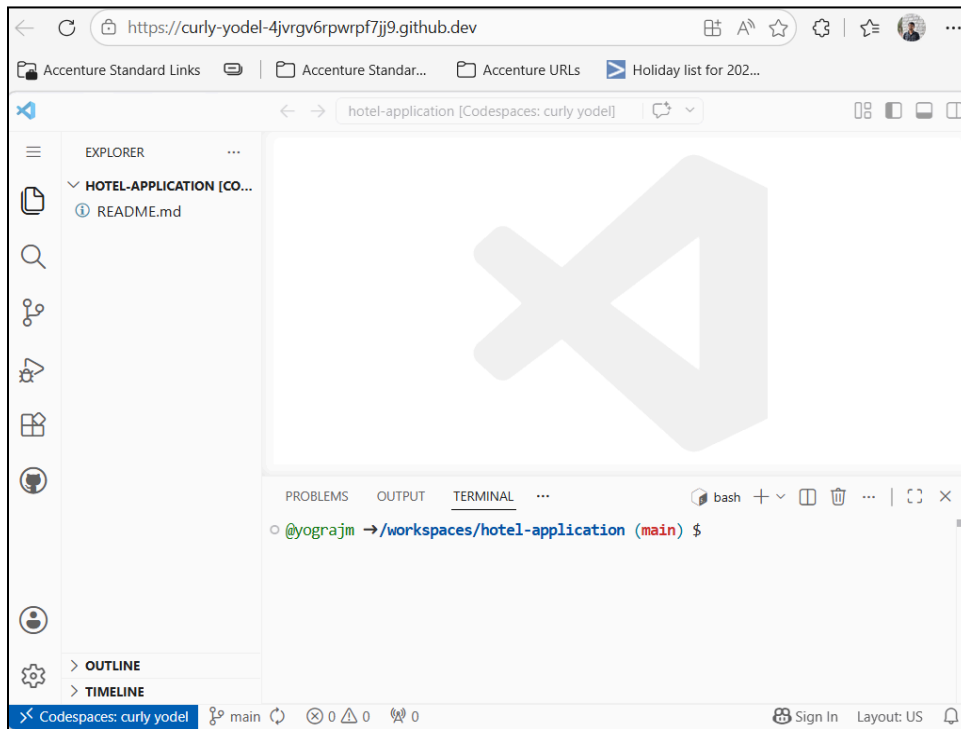
1. First, create an account on [GitHub](#) if you don't have one already.
2. Then go to GitHub Codespaces: <https://github.com/codespaces>
3. Start a new Codespace using the **Blank** template



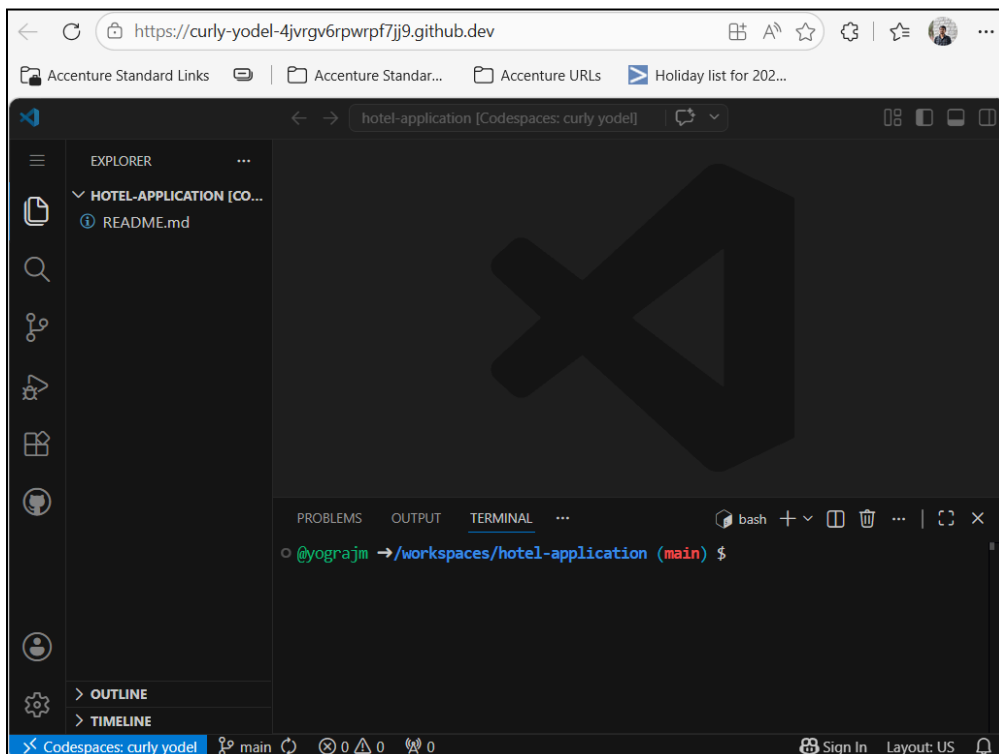
4. It will open a new tab, and start *Setting up your codespace*.



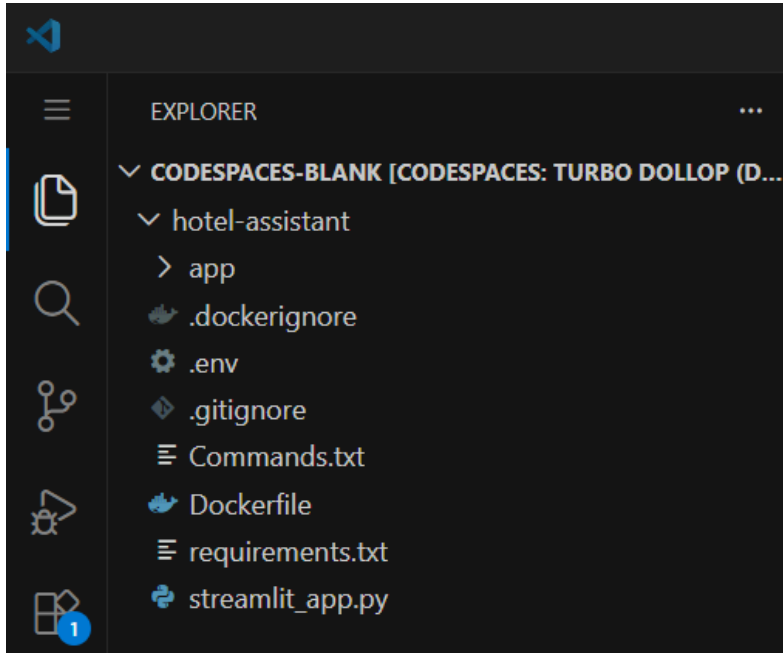
5. After loading, the Editor should open in the browser similar to below.



6. By default it is Light mode. You can change the theme by going to **Files -> Preferences -> Theme -> Color theme -> Default Dark Modern**.



7. Download the **hotel-assistant** folder zip file, shared along with this document, and unzip it in your system. Upload the entire folder to your GitHub Codespace by drag & drop.
8. Application files should be visible on the GitHub Codespace.



Build the Docker Image

9. Next, review the Dockerfile

```
# Use official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /hotel-assistant

# Copy requirements file
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
```

```
COPY . .

# Expose Streamlit port
EXPOSE 8501

# Start both FastAPI backend and Streamlit frontend
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port 8000 &
streamlit run streamlit_app.py --server.port 8501 --server.address
0.0.0.0"]
```

10. Move inside the **hotel-assistant** folder and build the Docker Image.

```
cd hotel-assistant
```

```
docker build -t hotel-assistant-img .
```

The above command builds a Docker image from the Dockerfile present in the current directory.

Here, **-t hotel-assistant-img** gives the image a name/tag, and **.** tells Docker to use the current folder as the build context.

```
@yograjm →/workspaces/codespaces-blank $ cd hotel-assistant
@yograjm →/workspaces/codespaces-blank/hotel-assistant $
@yograjm →/workspaces/codespaces-blank/hotel-assistant $ docker build -t hotel-assistant-img .
[+] Building 37.5s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 540B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 8.7s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 58B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca 0.0s
=> [internal] load build context                                    0.0s
=> => transferring context: 1.26kB                                     0.0s
=> CACHED [2/5] WORKDIR /hotel-assistant                          0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [5/5] COPY . .                                                  0.1s
=> exporting to image                                              28.5s
=> => exporting layers                                              0.1s
=> => exporting manifest sha256:15862f4d942c480cdb6d20699f00a514676a902721df1c9f63cdc2ec43cef088 0.0s
=> => exporting config sha256:0d00aea53800ed5b08b8bb7e0ac0cabb856efe0c92e615ff08eb4c321771baff 0.0s
=> => exporting attestation manifest sha256:57a0e271ef6ae6201fd9957677e6cb6bfb5523127c63323bca6473389a0f6b72 0.0s
=> => exporting manifest list sha256:09ae494f5929c2a15439bb455c875ee63aae048a1a161bf92a29df3c021ec0dc 0.0s
=> => naming to docker.io/library/hotel-assistant-img:latest      0.0s
=> => unpacking to docker.io/library/hotel-assistant-img:latest   28.3s
```

11. Check the Docker Image created

```
docker images
```

This lists all Docker images available locally on your system.



It shows details such as the image name, tag, image ID, creation time, and size.

```
@yograjm → /workspaces/codespaces-blank/hotel-assistant $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hotel-assistant-img:latest	09ae494f5929	1GB	220MB	

Run a Docker Container

12. Add the OpenAI API Key / Groq API Key in the **.env** file

13. Run a Docker Container using the created Docker Image

```
docker run -it -d -p 8501:8501 --env-file .env --name hotel-app hotel-assistant-img
```

Here, the container will run

- with a pseudo terminal (**-it**)
- in a detached mode (**-d**)
- with container port mapped to host port (**-p 8501:8501**)
- as a root user (**-u root**)
- With **.env** file loaded using **--env-file** argument
- with name hotel-app (**--name hotel-app**)
- using the hotel assistant docker image (**hotel-assistant-img**)

```
@yograjm → /workspaces/codespaces-blank/hotel-assistant $ docker run -itd -p 8501:8501 --env-file .env --name hotel-app hotel-assistant-img
59c6a53e690e0b7e1fc91f16aa0da3f46b9941667976c4ddcbbef553a881daba
```

14. Check the Docker Container if its running

```
docker ps
```

This lists all currently running Docker containers.

It shows details such as container ID, image name, command, status, ports, and container name.

```
@yograjm → /workspaces/codespaces-blank/hotel-assistant $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
59c6a53e690e	hotel-assistant-img	"sh -c 'uvicorn app....'"	6 minutes ago	Up 6 minutes	0.0.0.0:8501->8501/tcp,
8501->8501/tcp	hotel-app				

15. Check the logs of the Container. The application must be running

```
docker logs -f hotel-app
```



This shows the real-time logs of the running container named `hotel-app`.
Here, `-f` means **follow**, so the terminal keeps streaming new logs continuously as the application runs.
(Press `Ctrl+C` to close the logs.)

```
@yograjm → /workspaces/codespaces-blank/hotel-assistant $ docker logs -f hotel-app

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

2026-06-17 11:28:24.439 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

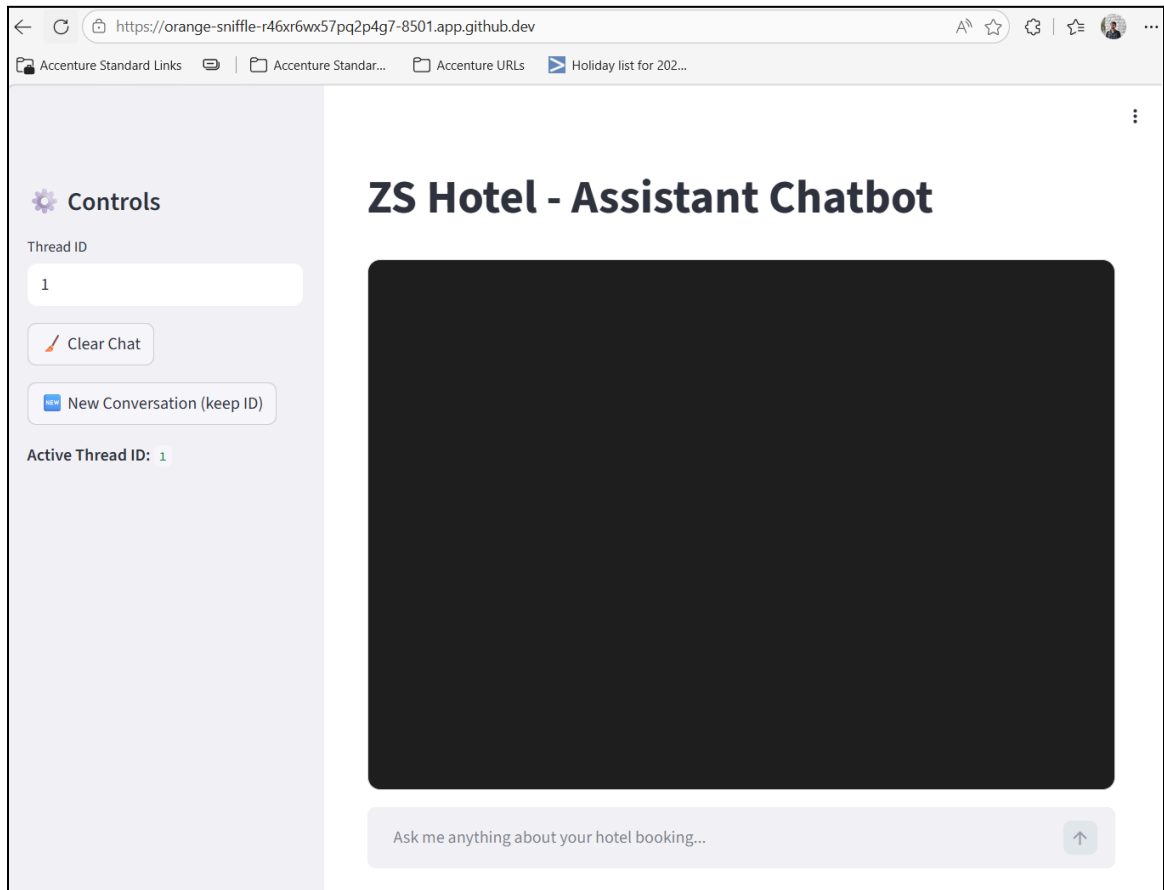
Local URL: http://localhost:8501
Network URL: http://172.17.0.2:8501
External URL: http://13.71.3.96:8501

INFO:      Started server process [7]
INFO:      Waiting for application startup.
Hotel DB initialized.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

16. Access the application by opening the forwarded address of the GitHub Codespace for port 8501

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1			
Port	Forwarded Address	Running Process	Visibility
8501	https://turbo-dollop-jj7g479g...	sh -c uvicorn app.main:app --host ...	Private
Add Port			

17. Once the port is forwarded, you can access its 'Forwarded Address' to access the application.



18. Try with some user inputs:

Input: *What rooms are available?*

ZS Hotel - Assistant Chatbot

What rooms are available?

We have the following rooms available: 101, 103, 104, 105, 201, and 203. Let me know if you need more information or if you'd like to book one!

Ask me anything about your hotel booking...



19. Stop the running container

```
docker stop hotel-app
```

```
• @yograjm → /workspaces/codespaces-blank/hotel-assistant $ docker stop hotel-app
hotel-app
• @yograjm → /workspaces/codespaces-blank/hotel-assistant $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
```

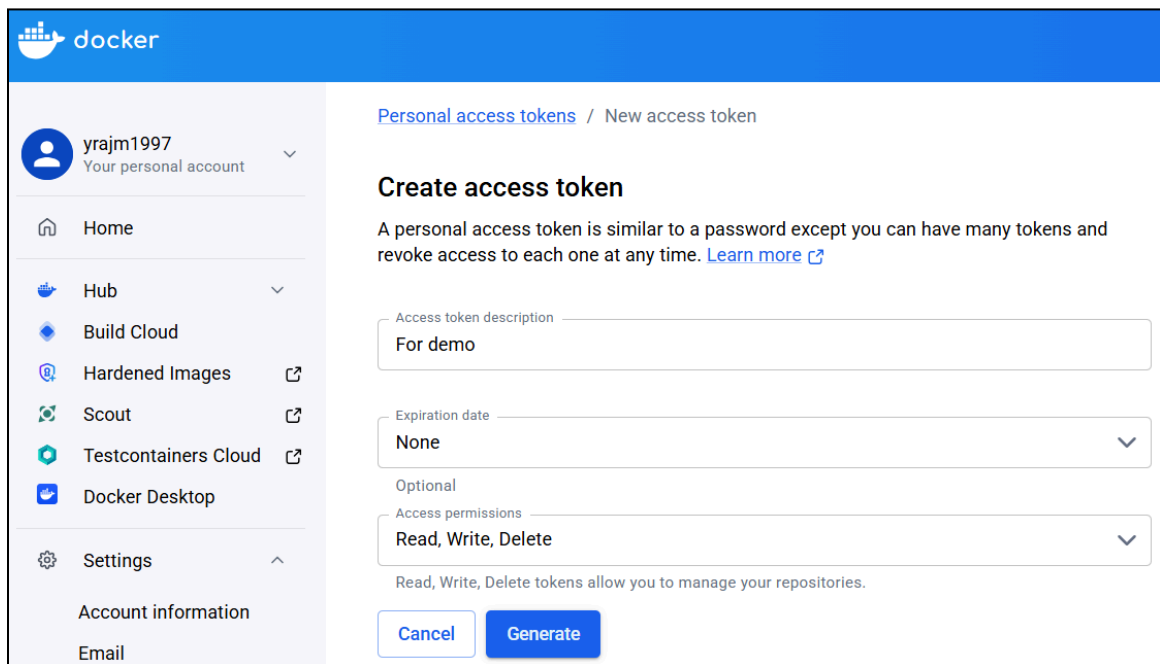
Push the Docker Image to Docker Hub

This means uploading your locally built Docker image to your Docker Hub account so that it can be accessed from anywhere.

This is useful when you want to deploy the image on platforms like Hugging Face Spaces, cloud servers, or share it with others without rebuilding it locally.

20. Create a Personal Access Token from your Docker Hub Account.

Visit: <https://hub.docker.com/> and Sign in. (Sign up for free if you don't have an existing account)
Go to your Profile >> Account settings >> Personal access tokens >> Generate new token
Provide a name for the token, and Access permissions: Read, Write, Delete >> Generate
Token must be generated.



To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u yraj
```

Copy

2. At the password prompt, enter the personal access token.

```
dckr_pat_C3
```

Copy

21. Login to Docker Hub from you Codespace

```
docker login -u <your-dockerhub-username>
```



This logs you in to Docker Hub from the terminal using your Docker Hub username.

After running this command, Docker will ask for your password or access token.

Paste your Personal Access Token when prompted.

NOTE that once you copy paste your token it will not show it in Terminal but it will be pasted.
Once authenticated, you can now push images to your Docker Hub account.

```
@yograjm →/workspaces/codespaces-blank/hotel-assistant $ docker login -u yrajm1997

Info → A Personal Access Token (PAT) can be used instead.
       To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/home/codespace/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

Tag your Docker Image

Tagging your Docker Image means giving your local image a proper repository name and version so it can be pushed to Docker Hub.

22. Tag the Docker Image that you built

Syntax: `docker tag image-name <your-dockerhub-username>/image-name:latest`

```
docker tag hotel-assistant-img
<your-dockerhub-username>/hotel-assistant-img:latest
```

Here, the image `hotel-assistant-img` is tagged with your Docker Hub username, repository name, and version tag `latest`.

```
@yograjm →/workspaces/codespaces-blank/hotel-assistant $ docker images

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
hotel-assistant-img:latest           09ae494f5929                     1GB          220MB        U

@yograjm →/workspaces/codespaces-blank/hotel-assistant $
@yograjm →/workspaces/codespaces-blank/hotel-assistant $ docker tag hotel-assistant-img yrajm1997/hotel-assistant-img:latest
@yograjm →/workspaces/codespaces-blank/hotel-assistant $
@yograjm →/workspaces/codespaces-blank/hotel-assistant $ docker images

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
hotel-assistant-img:latest           09ae494f5929                     1GB          220MB        U
yrajm1997/hotel-assistant-img:latest 09ae494f5929                     1GB          220MB        U
```

Push your Docker Image

23. Push the Docker Image from your system (Codespace) to Docker Hub.

Syntax: `docker push <your-dockerhub-username>/image-name:latest`

`docker push <your-dockerhub-username>/hotel-assistant-img:latest`

```
@yograjm → /workspaces/codespaces-blank/hotel-assistant $ docker push yrajm1997/hotel-assistant-img:latest
The push refers to repository [docker.io/yrajm1997/hotel-assistant-img]
84e497fde9e0: Pushed
1c9641dee63c: Pushed
8c471a93fc52: Layer already exists
3250ea7ceadc: Layer already exists
3c7487f82677: Layer already exists
72b853727ece: Layer already exists
4326b3452143: Layer already exists
72c03230f136: Layer already exists
8bd2cf7d3f6a: Layer already exists
latest: digest: sha256:09ae494f5929c2a15439bb455c875ee63aae048a1a161bf92a29df3c021ec0dc size: 856
```

24. Check the Docker Image pushed on your Docker Hub account.

[Repositories](#) / [hotel-assistant-img](#) / [General](#)

yrajm1997/hotel-assistant-img

Last pushed 1 minute ago • Repository size: 210.1 MB • ☆0 • ↓33

[Add a description](#)

[Add a category](#)

[General](#) [Tags](#) [Image Management](#) [Collaborators](#) [Webhooks](#) [Settings](#)

Tags

DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	1 minute

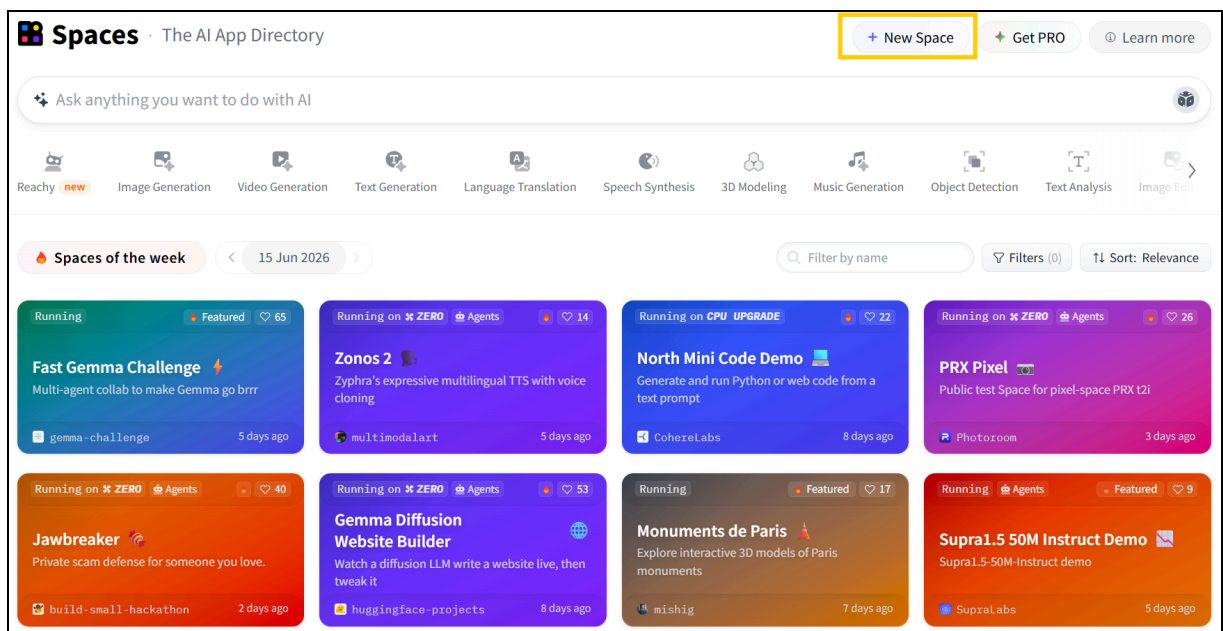
Deploy the Application on Hugging Face Spaces

This means hosting your Docker-based application on Hugging Face so users can access it through a public web URL.

For this, create a new Space with **SDK = Docker**, add required files like `Dockerfile`, `requirements.txt`, and app code, configure secrets such as `OPENAI_API_KEY`, and push the project files to the Space repository.

25. Go to the [Hugging Face Hub](https://huggingface.co) and login.

26. Go to the [Spaces](https://huggingface.co/spaces), you will see the spaces created by the community.



27. To create a new Space, select **New Space**.

28. Provide Space details like

a. Space name: *HotelApp*

Create a new Space

Spaces are Git repositories that host application code for Machine Learning demos.
You can build Spaces with Python libraries like Gradio, or using Docker images.

Owner: / Space name:

Short description:

- b. Select the Space SDK as **Docker** and Choose the Docker template as **Blank**
The Hub offers three SDK options: Gradio, Docker, and Static HTML.

Select the Space SDK

You can choose between Gradio, Docker, or Static to host your Space.

Gradio
4 templates

Docker
17 templates

Static
6 templates

Choose a Docker template:

Blank

Streamlit

JupyterLab

Argilla

Livebook

LabelStudio

AimStack

Shiny (R)

- c. Space Hardware: select the FREE one - **CPU Basic**

Space hardware

Free

CPU Basic

ZeroGPU

Custom Hardware

You can switch to a different hardware at any time in your Space settings.

- d. Visibility: **Public** (can be changed later through Space Settings)



☒ **Public**
Anyone on the internet can see this Space. Only you (personal Space) or members of your organization (organization Space) can commit.

☐ **Private**
Only you (personal Space) or members of your organization (organization Space) can see and commit to this Space.

Create Space

e. Select **Create Space**.

29. Your Hugging Face Space Repository should be created. Go to the **Files** tab.

huggingface.co/spaces/yograjm/HotelApp

Accenture Standard Links

Spaces yograjm/HotelApp like 0 No application file App **Files** Community

Hugging Face is way more fun with friends and colleagues! 🥳 [Join an organization](#)

Get started with your Docker Space!
Your new Space has been created, follow these steps to get started (or read the full [documentation](#))

Start by cloning this repo by using:

HTTPS SSH

Use an [access token](#)

When prompted for a password, use an access token with write permissions.
Generate one from your settings: <https://huggingface.co/settings/tokens>
git clone https://huggingface.co/spaces/yograjm/HotelApp

30. In the Files tab, open the **README.md** file.

Spaces: yograjm/HotelApp like 0 No application file App **Files** Community Settings

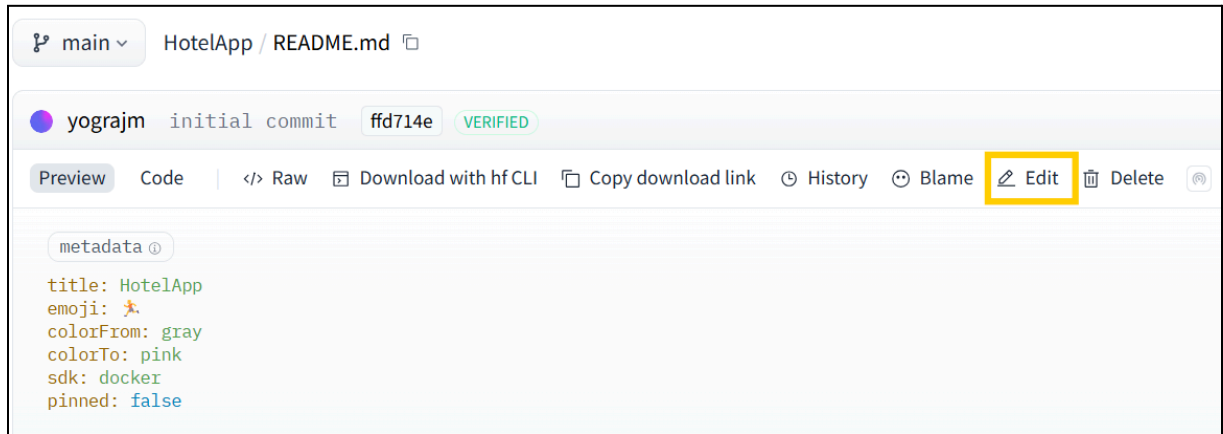
main HotelApp 1.71 kB Go to file Ctrl+K 1 contributor History: 1 commit + Contribute

yograjm initial commit ffd714e VERIFIED 4 minutes ago

.gitattributes @ Safe 1.52 kB initial commit 4 minutes ago

README.md 190 Bytes initial commit 4 minutes ago

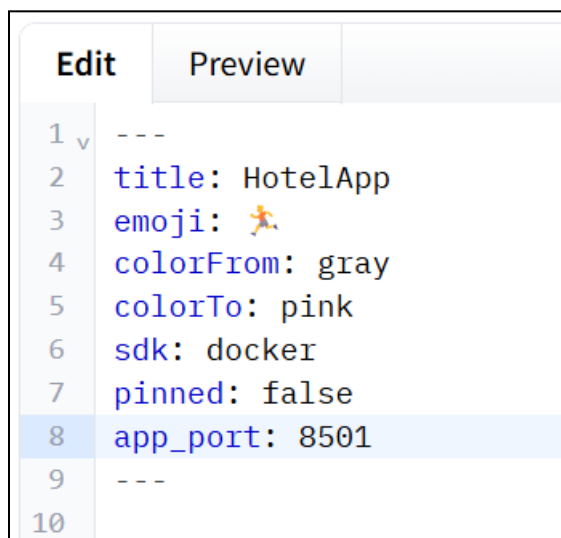
31. **Edit** the Readme file.



32. Add the App port line in the Readme file (**app_port: 8501**).

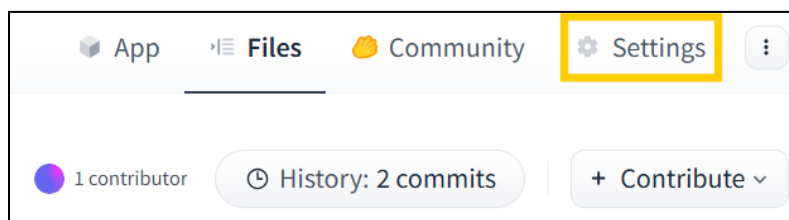
This is required as the application UI will be running on port 8501. By default, the Hugging Face Space expects the application to be running on port 7860.

Once done, commit the changes using the Commit button.



Save your OpenAI API Key / Groq API Key in your Hugging Face Space repository

33. Go to **Settings**



34. In the Settings, scroll to the **Variables and Secrets** and select **New Secret**.

Variables and secrets ⓘ

Variables Public

No variables

Secrets Private

No secrets

35. Add the key **Name** and **Value**, then **Save**.

New secret

Name

OPENAI_API_KEY

Description (optional)

Description

Value (private)

sk-proj-
e1Q46JYH6h87gHIXInvz0t6asx8W26ED_xbNjVcHxehZabj3F8nBzW58Z0Mcwpg
...
...

Save Cancel

36. The **Secret** should be added.

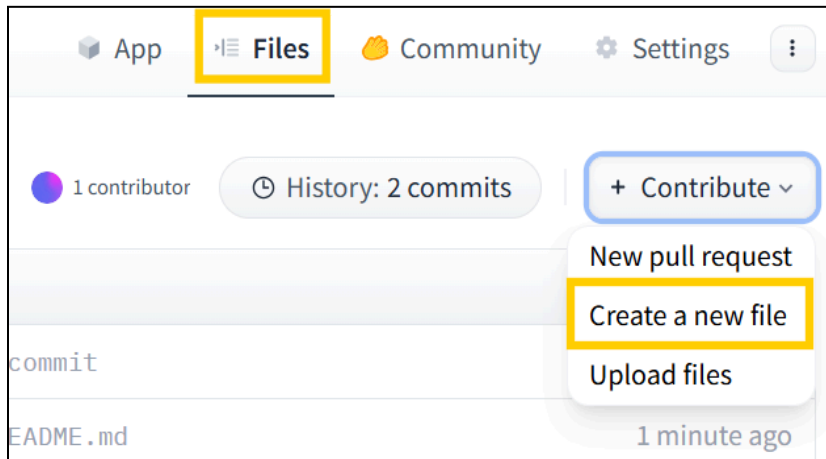
Secrets Private

OPENAI_API_KEY · Updated less than a minute ago

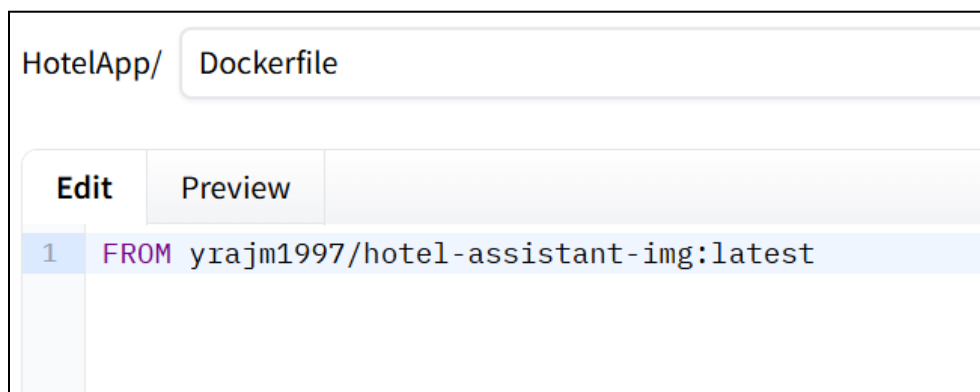
Replace Delete

Create a Dockerfile

37. Next, go to the **Files** tab again, then go to **Contribute**, and **Create a new file**.

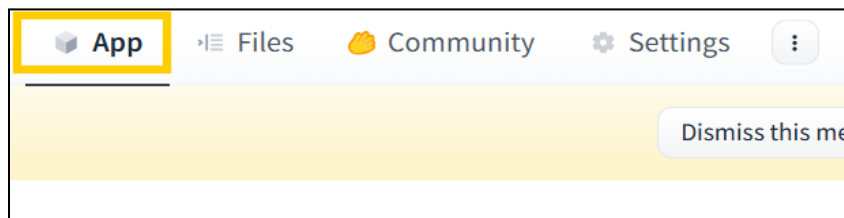


38. Add the file name **Dockerfile**, and in the content provide your Docker Image.
Once done, commit the changes using the Commit button.

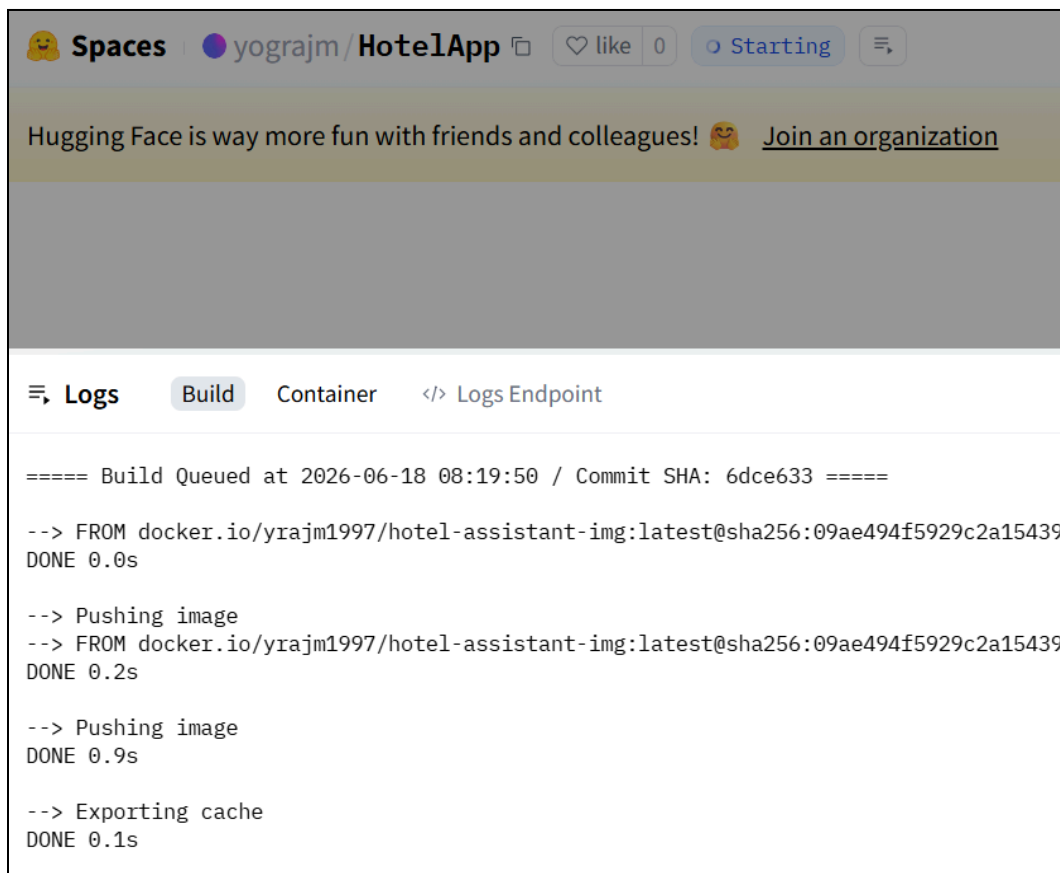
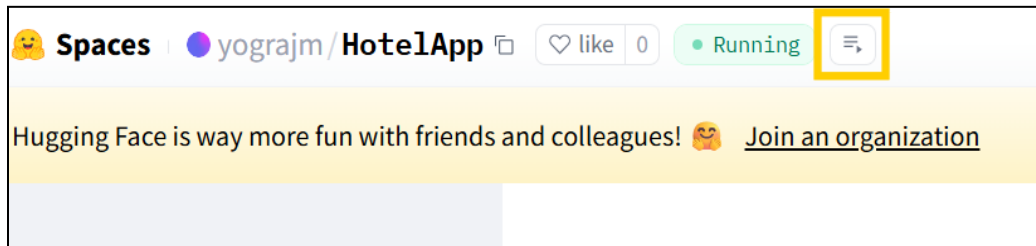


39. As soon as the Dockerfile is added, the application build will start.

Go to the **App** tab.



Open the **Logs** by clicking on the **Logs icon**.



After some time, the application will be up and running.

```
≡ Logs Build Container </> Logs Endpoint

===== Application Startup at 2026-06-18 08:20:00 =====

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

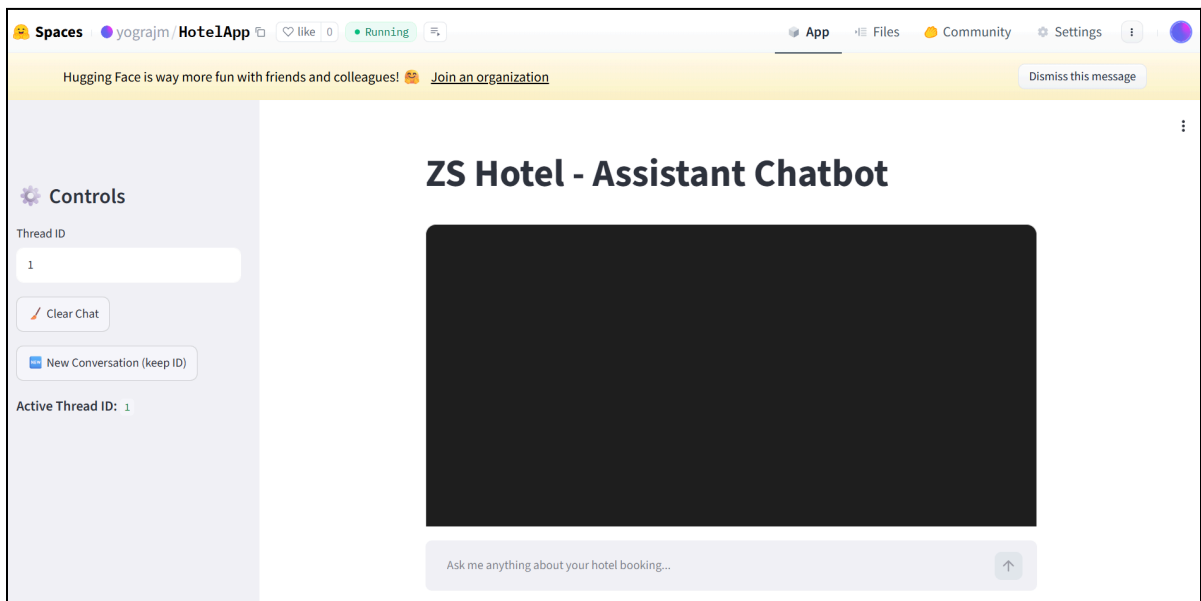
2026-06-18 08:20:22.103 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://10.112.2.131:8501
External URL: http://50.17.198.149:8501

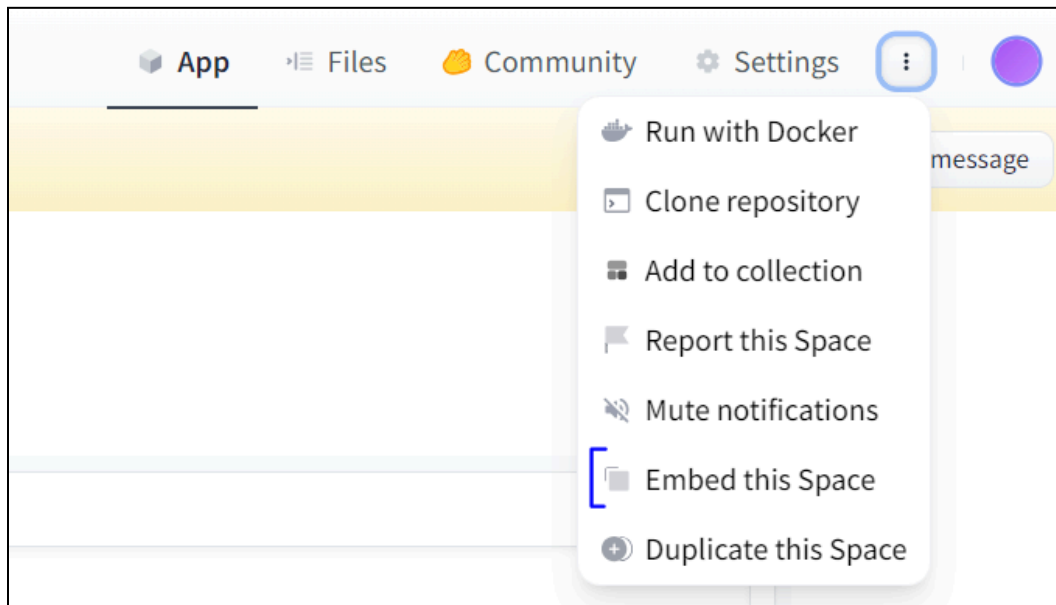
INFO: Started server process [7]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

40. Close the Logs, and the application UI should be visible.

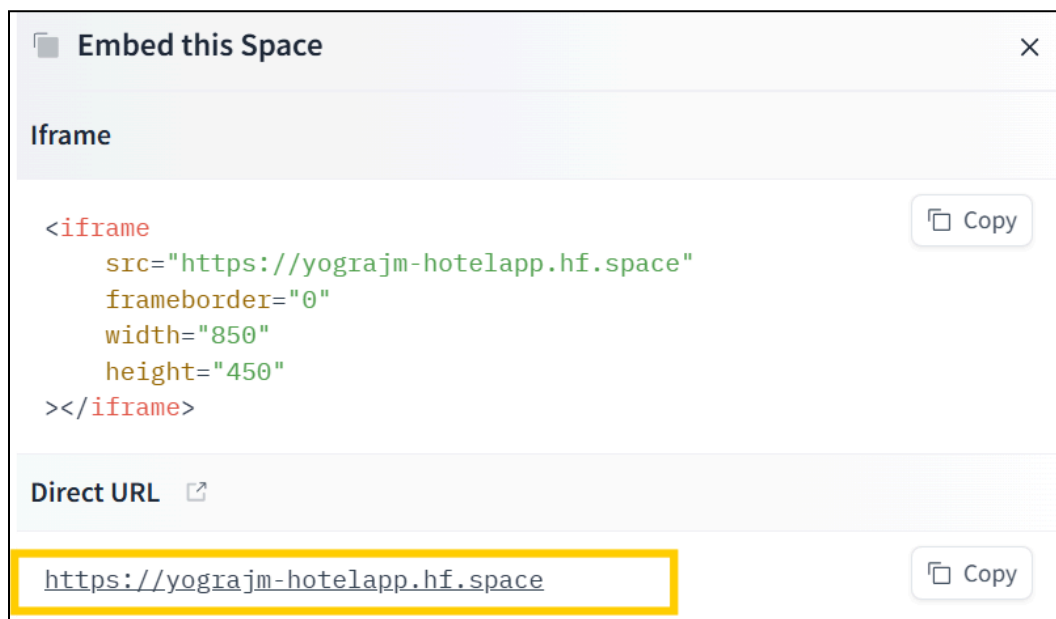


41. To get a shareable link for your application, go to the 3 dots icon beside **Settings** and select **Embed this Space**.

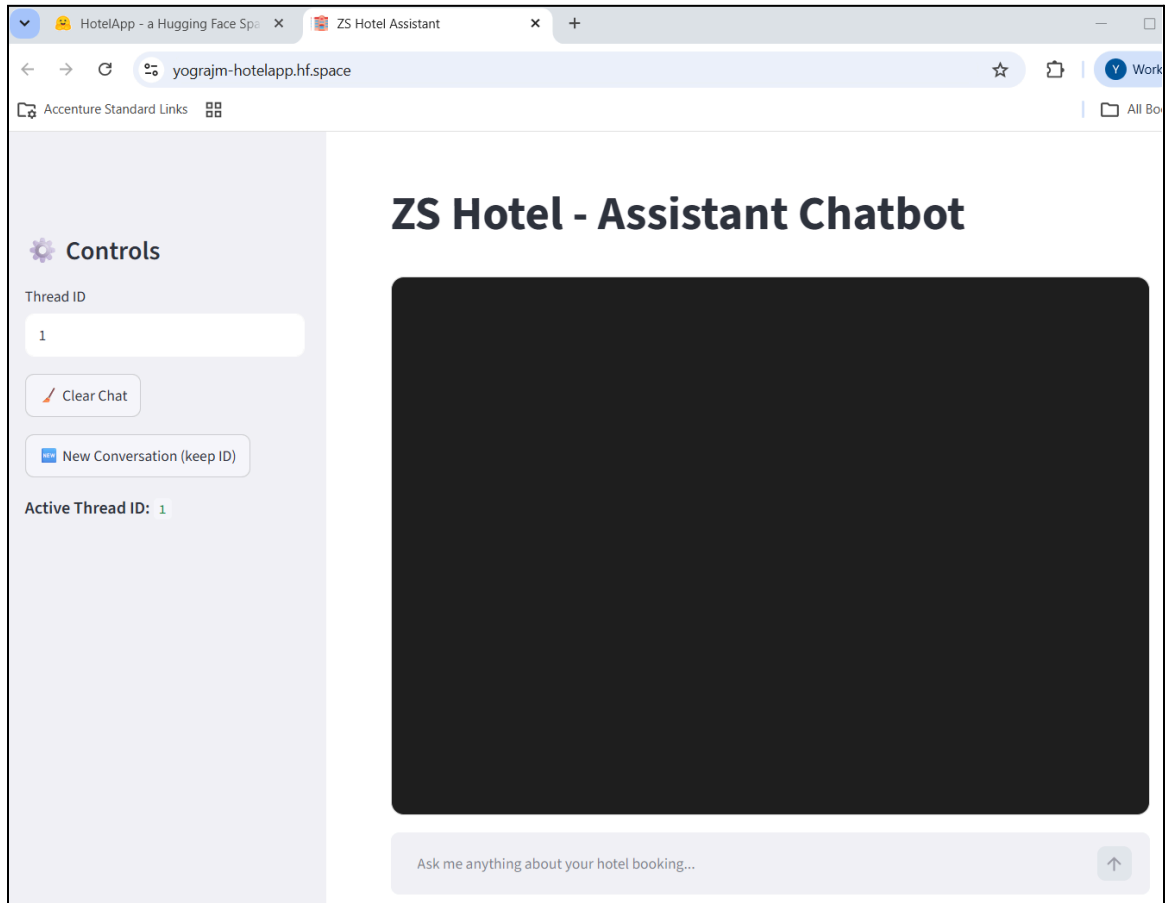
NOTE: 'Embed this Space' option will come only if your repo is **Public**. If it is a Private repo, you can update the visibility to Public through Settings.



42. You will get a Direct URL for your application.



43. Your application should be accessible through this Direct URL.



44. Try with some user inputs:

Input: *What rooms are available?*

Input: *Book room 105*



ZS Hotel - Assistant Chatbot

What rooms are available?

We have the following rooms available: 101, 103, 104, 105, 201, and 203. Let me know if you need more information or if you'd like to book one!

Book room 105

Could you please provide the name under which you'd like to book room 105?

Ask me anything about your hotel booking...



Input: *Bharath*

ZS Hotel - Assistant Chatbot

What rooms are available?

We have the following rooms available: 101, 103, 104, 105, 201, and 203. Let me know if you need more information or if you'd like to book one!

Book room 105

Could you please provide the name under which you'd like to book room 105?

Bharath

Room 105 is now successfully booked for Bharath. If there's anything else you need, feel free to ask!

Ask me anything about your hotel booking...



Input: *Book room 103 for Srikanth*

Input: *Book room 103 for Ramesh*

Input: *Which rooms are available?*

Input: *Book room 203 for Ramesh*

Input: *what south indian food is available*

Input: *I want to order 2 pav bhaji*

Input: *103*

Input: *Order 3 South Indian Thali to room 105*

Input: *Order 2 servings of Pasta and 1 Noodles to room 105*

Input: *I want food bill for 105*

45. Note that as the application is **Public**, anyone can interact with it. So once you are done, make your application **Private**, to prevent unwanted cost.

~~~END~~~